Лабораторна робота N3

Масиви та колекції. Клонування. Глибинне та поверхневе копіювання. Інтерфейси.

Задача. Навчитись використовувати масиви та колекції, клонування та глибинне копіювання. Навчитись використовувати стандартні інтерфейси Comparable, Comparator та Cloneable.

Завдання.

0. Затвердити тему курсової роботи. Якщо студент ухиляється від розробки та подання власної теми, її призначає викладач. Затверджені теми розміщуються в «Навігаторі дисципліни». До моменту публікації фінального файлу статус тем у тимчасовому переліку позначається кольорами:

- **Чорний колір** — теми, що запропоновані студентами та успішно затверджені викладачем;
- **Зелений колір** — теми, призначені викладачем, прирівнюються до затверджених (студент має тиждень, щоб запропонувати власну альтернативу);
- **Червоний колір** — зафіксовані теми, що потребують подальшого обговорення та виправлення.

Захист лабораторної роботи №3 за незатвердженою (червоною) темою не допускається!

1. Створити консольний проект. З лістингу програми до лаб.роботи №2 перенести в проект лаб. роботи №3 клас мікрооб'єкта. Додати до програми клас універсального об'єкта. Призначення універсального об'єкта у тому, щоб містити всі інші об'єкти програми. В якості універсального об'єкта можна використовувати клас Main.

2. Додати до класу мікрооб'єкта в якості елемента посилальний тип, щоб зробити необхідним глибинне копіювання при клонуванні.

3. Запитати в користувача кількість об'єктів. В універсальному об'єкті створити масив другого типу (Масив другого типу – це масив посилань на об'єкти – це єдиний можливий тип масива об'єктів у Java). Далі в тексті цей масив має назву **поточний масив**.

4. Створити меню з команд
   4.1. Вивести всі об'єкти поточного масиву у зручній формі (по одному в рядку).

4.2. Передивитись і-й об’єкт поточного масиву.
4.3. Додати на і-у позицію у поточний масив об’єкт (запитати параметри об’єкта). Мається на увазі, що об’єкт, який перед цим займав і-ту позицію, посунеться і буде обіймати і+1-шу позицію і так далі. Кількість елементів у масиві при цьому збільшиться на один.
4.4. Видалити і-й об’єкт поточного масиву.
4.5. Відсортувати за певною ознакою об’єкти поточного масиву об’єктів. Повинна бути можливість відсортувати поточний масив об’єктів як мінімум за двома ознаками (наприклад: Ім’я_студента та Середній_бал). З 4-х способів організації компараторів:
1) засобами інтерфейсу Comparable;
2) вкладеним класом (nested class) класу мікрооб’єкта;
3) анонімним класом;
4) посиланням на метод (method reference);
обрати два, номера яких вказані в Таблиці 2 згідно варіанту.

4.6. Реалізувати глибоке копіювання для класу мікрооб’єкта. За командою з меню програма має запитати у користувача індекс *i* об’єкта та додати його копію в кінець поточного масиву. Для демонстрації роботи глибокого копіювання необхідно змінити дані в оригінальному *i*-му об’єкті, після чого вивести на екран стани обох об’єктів (оригіналу та копії) для порівняння.

4.7. Реалізувати пошук об’єкта у поточному масиві з допомогою функції Arrays.binarySearch. Для цього відсортувати масив згідно критерію пошуку по зростанню. Запитати в користувача параметри пошуку. Створити тимчасовий об'єкт з параметрами пошуку. Здійснити пошук тимчасового об'єкта в поточному масиві функцією Arrays.binarySearch і вивести на екран результати пошуку. Факультативне завдання: якщо в масиві міститься більше одного об’єкта, який задовольняє параметрам пошуку, то видати на екран позиції в масиві всіх подібних об’єктів.

4.8. Додати в меню дві команди, кожна з яких виконує один пункт з Таблиці 1. Номера пунктів з Таблиці 1, які слід реалізувати, знаходяться з Таблиці 2 згідно варіанту.

Таблиця 1 – Функціонал п 4.8 меню

№	Функціонал
1	З об’єктів поточного масиву утворити тимчасовий масив. шляхом глибинного копіювання. Утворений тимчасовий масив вивести на екран, якимось чином змінивши 0-й елемент масиву. Після цього користувач може обрати п. 4.1 з меню і подивитись, чи змінився 0-й об’єкт поточного масиву
2	Запитавши параметри в користувача створити тимчасовий об’єкт. Тимчасовий об’єкт повинен взаємодіяти послідовно з об’єктами поточного масиву, при цьому на екран має видаватись повідомлення про характер взаємодії і її результат
3	Запитати в користувача певний параметр, який дозволяє поділити об’єкти поточного масиву на дві категорії. Після запиту в користувача видалити з поточного масиву об’єкти однієї з двох категорій.
4	Якимось чином з об’єктів поточного масиву утворити два тимчасові масиви (дві “команди”) і провести між ними “змагання”. Вивести на екран склади команд і результат змагання. Наприклад в одну “команду” можна помістити об’єкти з непарним номером, а в іншу - об’єкти з парним номером. Чи, наприклад, утворити першу “команду” з об’єктів першої половини поточного масиву, а другу “ команду” - з об’єктів другої половини поточного масиву. Наприклад, якщо мікрооб’єктами є Зомбі, то в ході “змагання” сильніші Зомбі поїдають слабших. Програма має вивести на екран повідомлення типу “Перша команда виграла” чи “Друга команда виграла” чи “Нічія”. Відповідь на питання “Чи змінюється в результаті змагань поточний масив” залишається за програмістом. Якщо об'єктів непарна кількість і це заважає утворенню “команд” повідомити про це користувачу.

Таблиця 2 – Варіанти завдань

Номер варіанта	Реалізація компараторів п. 4.5		Функціонал команд з п. 4.8	
1	2	3	1	3
2	1	2	3	4
3	3	4	2	4
4	2	4	2	3
5	2	3	1	2
6	3	4	3	4
7	2	3	1	4
8	2	3	2	3
9	3	4	2	3
10	1	3	1	4
11	1	2	1	4
12	2	4	3	4
13	1	4	1	4
14	2	3	2	4
15	2	4	1	4
16	1	3	1	2
17	3	4	1	3
18	1	4	2	4
19	1	2	2	3
20	1	2	1	3
21	1	4	1	3
22	1	4	2	3
23	2	3	3	4
24	1	3	2	4

25	1	3	1	3
26	1	4	1	2
27	1	3	3	4
28	2	4	1	2
29	2	4	1	3
30	1	4	3	4
31	1	2	1	2
32	1	2	2	4
33	2	4	2	4
34	1	3	2	3
35	3	4	1	4
36	3	4	1	2

5. Додати до проекту два макрооб’єкта. Кожен макрооб’єкт повинен містити масив мікрооб’єктів.
6. Додати до меню наступні команди:
   6.1 Додати мікрооб’єкт. Програма має запитувати параметри мікрооб’єкта та куди його слід додати: в один з макрооб'єктів чи в поточний масив в універсальному об’єкті.
   6.2 Вивести на екран вміст універсального об’єкта та всіх макрооб’єктів. Список мікрооб’єктів повинен виводитись зручно – по одному мікрооб’єкту в рядок.
   6.3 Взаємодія мікрооб’єктів. Програма має запитати в користувача критерій за яким відбираються два (або більше) мікрооб’єкта для взаємодії. Мікрооб'єкти для взаємодії можуть обиратись як з тих, що містяться в середині макрооб’єктів, так і з тих, які зберігаються в поточному масиві. Після цього програма має повідомити яка саме взаємодія відбулась і її результат. В результаті взаємодії мікрооб’єктів вони можуть змінюватись, знищуватись, переходити між макрооб’єктами та універсальним об’єктом. Також можуть з’являтись нові мікрооб'єкти в одному з макрооб'єктів чи в універсальному об’єкті.
   6.4 Взаємодія двох макрооб’єктів. При виборі цієї команди взаємодіють два макрооб'єкти шляхом взаємодії множин мікрооб’єктів, які вони містять. В результаті взаємодії макрооб’єктів вони можуть змінюватись самі так і

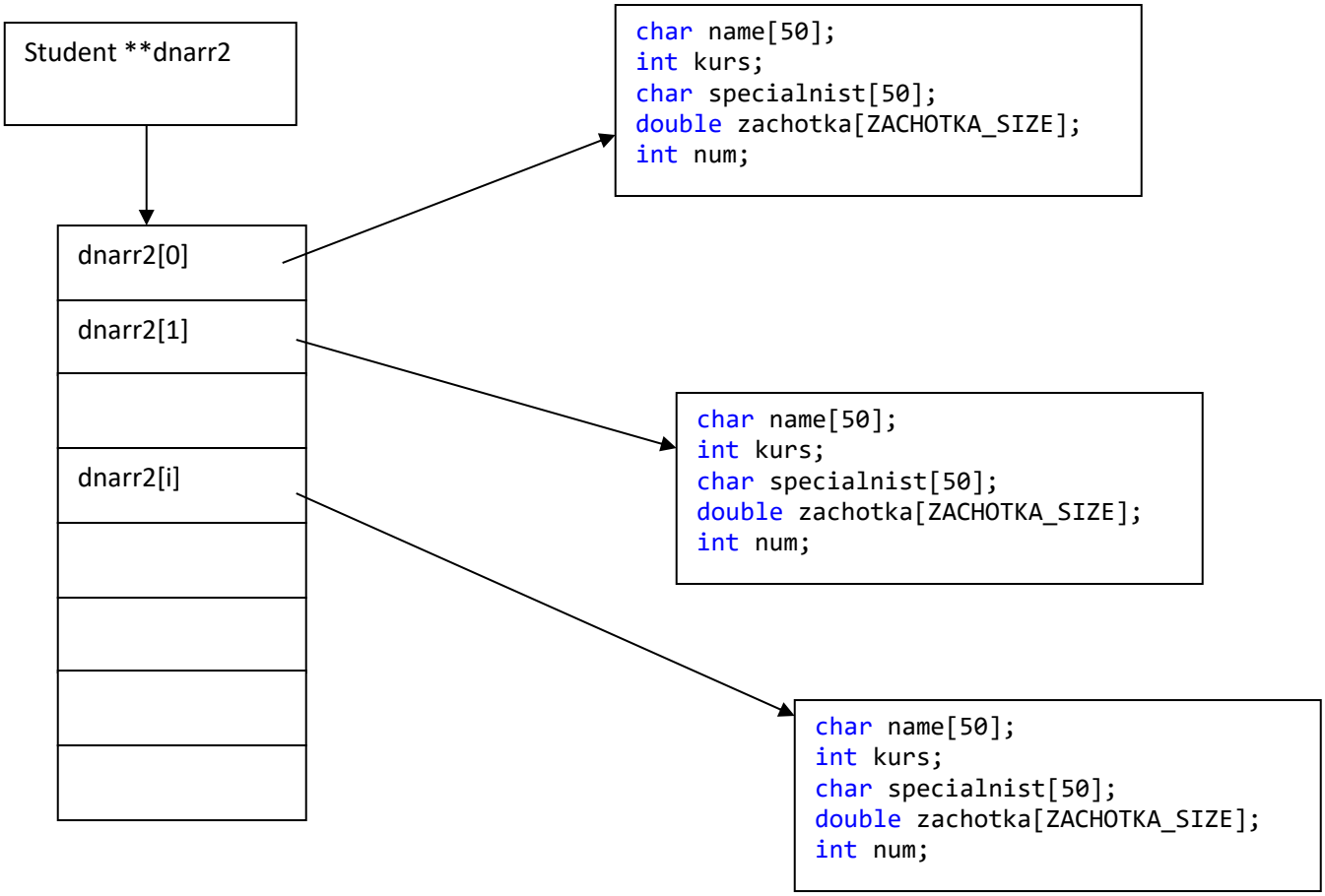
змінювати стан мікрооб’єктів, які в них містяться. Деякі мікрооб’єкти можуть знищуватись, переходити між макрооб’єктами та універсальним об’єктом. Також можуть з’являтись нові мікрооб'єкти в одному з макрооб'єктів чи в універсальному об’єкті.

6.5 Підрахунок мікрооб’єктів. При виборі цієї команди програма має запитати один з критеріїв підрахунку (всього таких критеріїв має бути не менше трьох) і після цього видати на екран інформацію про кількість мікрооб’єктів, які відповідають критерію, та до яких макрооб’єктів чи універсального об’єкта вони належать.

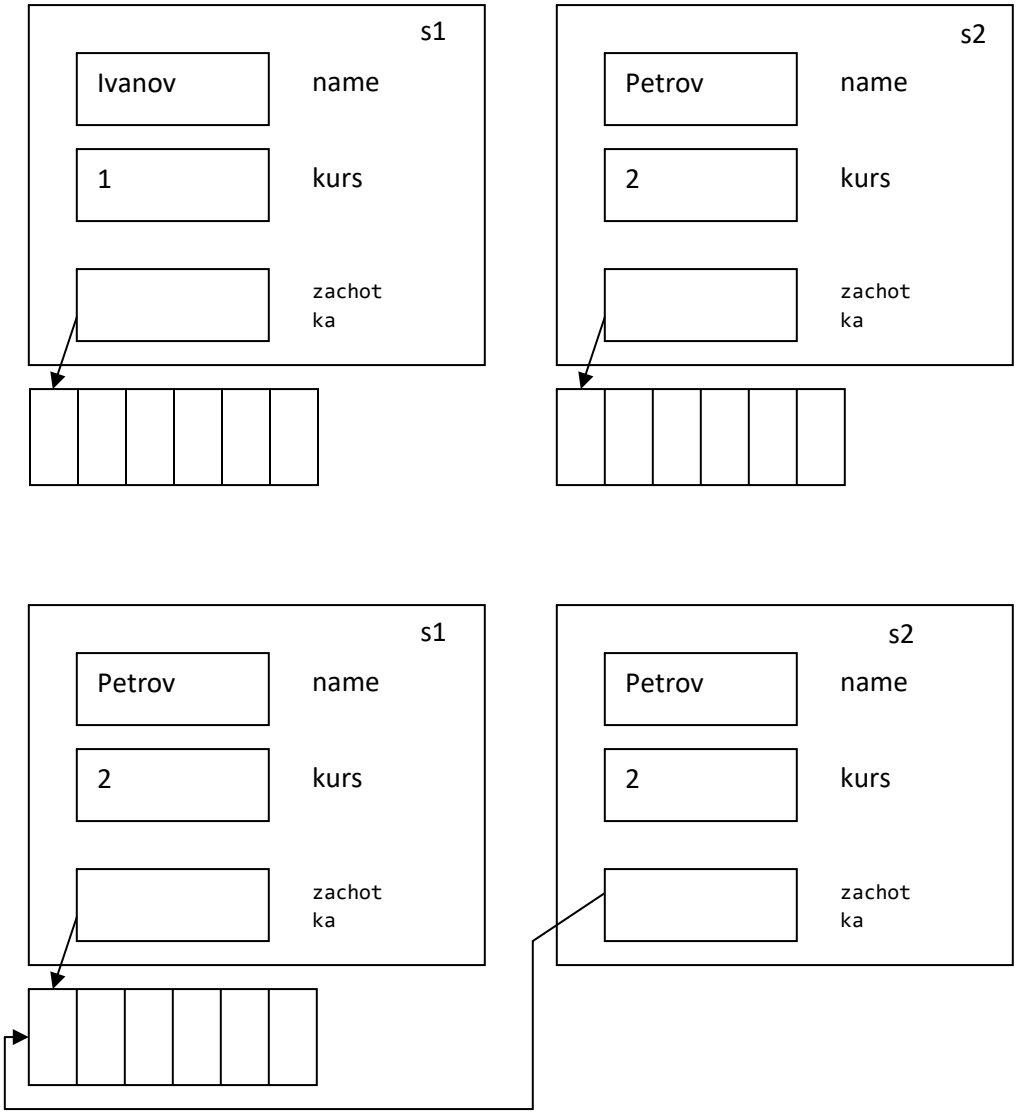
6.6 Видалити мікрооб’єкт. Програма має запитати, з якого з макрооб'єктів чи з універсального об’єкта слід видалити мікрооб’єкт і якось його ідентифікувати (критерій на вибір студента). Після цього програма має видалити вказаний мікрооб’єкт.

Теоретичні відомості

Наступний рисунок ілюструє динамічно створений масив об’єктів другого типу (у мові C++)



Наступний рисунок ілюструє виникнення прблеми при поверхневому копіюванні деяких об’єктів



Interface	Abstract
Java interface are implicitly abstract and cannot have implementations	A Java abstract class can have instance methods that implements a default behavior
Variables declared in a Java interface is by default final	An abstract class may contain non-final variables.
Members of a Java interface are public by default	A Java abstract class can have the usual flavors of class members like private, protected, etc
Java interface should be implemented using keyword "implements"	A Java abstract class should be extended using keyword "extends"
An interface can extend another Java interface only	an abstract class can extend another Java class and implement multiple Java interfaces.
Interface is absolutely abstract and cannot be instantiated	A Java abstract class also cannot be instantiated, but can be invoked if a main() exists.
java interfaces are slow as it requires extra indirection	Comparatively fast

```
/*

/*
Наступний проект.
Він демонструє глибинне копіювання.
 */
// TestCloning.java
public class TestCloning {

    public static void main(String[] args) throws CloneNotSupportedException {
        Department hr = new Department(1, "Human Resource");
        Employee original = new Employee(1,
                "Admin", hr);
        Employee cloned = (Employee) original.clone();

        //Let change the department name in cloned object and we will verify in
original object
        cloned.getDepartment().setName("Finance");

        System.out.println(original.getDepartment().getName());
        System.out.println(cloned.getDepartment().getName());
    }
}
```

```
// Employee.java
public class Employee implements Cloneable {

    private int empoyeeId;
    private String employeeName;
    private Department department;

    public void setDepartment(Department d)
    {
        department=d;
```

```
    }

    public Department getDepartment()
    {
        return  department;
    }

    public Employee(int id, String name, Department dept)
    {
        this.empoyeeId = id;
        this.employeeName = name;
        this.department = dept;
    }

  /*
//Поверхневе копіювання
    @Override
    protected Object clone()  throws CloneNotSupportedException
    {
        return super.clone();
    }
*/

//Глибинне копіювання
    @Override
    protected Object clone() throws CloneNotSupportedException
    {
        Employee cloned = (Employee)super.clone();
        cloned.setDepartment
                ((Department)this.getDepartment()
                        .clone());
        return cloned;
    }
}
```

```
// Department.java
public class Department implements Cloneable
{
    private int id;       //Примітивний тип
    private String name;  //Специфічний посилальний тип - immutable.

    public Department(int id, String name)
    {
        this.id = id;
        this.name = name;
    }

    //Getters and Setters
    public void setName(String s)
    {
        name =s;
    }
    public String getName()
    {
        return name;
    }

    //Як не дивно вистачає поверхневого копіювання!
    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
```

```
    }
}
```

Пояснення з сайту StackOverflow

https://stackoverflow.com/questions/1552301/immutability-of-strings-in-java

чому немає проблем зі злипанням String (посилального типу). Саме через це для String достатньо поверхневого копіювання.

```
String s1 = "Hello";
String s2 = s1;
// s1 and s2 now point at the same string - "Hello"
```

Now, there is nothing[1] we could do to `s1` that would affect the value of `s2`. They refer to the same object - the string `"Hello"` - but that object is immutable and thus cannot be altered.

```
/*
Наступний проект ілюструє відмінність примітивних типів від посилальних
 */
//Main.java

package com.company;

public class Main {
    public static void main(String[] args ) throws CloneNotSupportedException
    {
       Student s = new Student("Obama",3);


        System.out.println(s);
        s.Print();

        //Primitie types vs Reference
        //1
        int a=10;
        System.out.println(a);

        Student t;
        t= new Student("Trump",5);

        //2
        int b=20;
        System.out.println(a+" "+b);
        a=b;
        System.out.println(a+" "+b);
        b=100;
        System.out.println(a+" "+b);

        s.exam(5.0);

        t=s;
```

```
System.out.println(s);
System.out.println(t);

t.exam(2.0);
t.exam(2.0);
t.exam(3.0);

System.out.println(s);
System.out.println(t);

//3

a=10; b=10;

if( a==b) System.out.println("a==b!!!");
else  System.out.println("a!=b???");

s= new Student("Obama",1);
t= new Student("Obama",1);

if( s==t) System.out.println("s==t!!!");
else  System.out.println("s!=t???");

if( s.equals(t))System.out.println("s equals to t!!!");
else  System.out.println("s equals to t???");

//4

System.out.println("Before Change"+s);
Change(s);
System.out.println("After Change"+s);

s= new Student("Obama",1);
t= new Student("Trump",5);

System.out.println("Before Swap"+s);
System.out.println("Before Swap"+t);
Swap(s,t);
System.out.println("After Swap"+s);
System.out.println("After Swap"+t);

//5
int []arr=new int[]{1,200,200,500,700};

Student []grupa = new Student[5];

grupa[0]=new Student("Makron",3);
grupa[1]=new Student("Erdogan",4);
grupa[2]=new Student("Merkel",5);
grupa[3]=new Student("Yanukovich",0);
grupa[4]=new Student("Yakubovich",2);

for(int i=0; i<arr.length; i++ )
{
    System.out.println( arr[i] + " " + grupa[i] );
}

s= new Student("Obama",1);
s.exam(3.0);
//t= new Student(s);
```

```
        t=(Student)s.clone();


        System.out.print("Before s=");
        s.Print();
        System.out.print("Before t=");
        t.Print();

        s.exam(4.0);
        s.exam(5.0);

        t.setName("Barak");

        System.out.print("After s=");
        s.Print();
        System.out.print("After t=");
        t.Print();

//        int a=10, b=20;
//        Integer t=a;
//        Integer r=b;
//        System.out.println(a+" "+b);
//        Swap(t,r);
//        System.out.println(t+" "+r);

    }

    public static void Swap( Student a, Student b )
    {
        Student tmp=a; a=b; b=tmp;

        String tmps= a.getName();
        a.setName(b.getName());
        b.setName(tmps);

        int tmpk = a.getKurs();
        a.setKurs(b.getKurs());
        b.setKurs(tmpk);

    }

    public static void Change( Student st )
    {


        st=new Student("Trump",5);

        st.exam(2.0);
        st.exam(2.0);
        st.exam(2.0);
    }


    public static void Swap( Integer x, Integer y)
    {
        Integer tmp = x;
        x=y;
        y=tmp ;
    }
```

```
}

//Student.java
package com.company;

import java.util.Arrays;
import java.util.Objects;

public class Student implements Cloneable {

    private String name;
    private int kurs;
    private double [] zachotka;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getKurs() {
        return kurs;
    }

    public void setKurs(int kurs) {
        this.kurs = kurs;
    }

    public static String University="VNTU";

    public void exam(double ocinka) {
        if (zachotka == null) {
            zachotka = new double[1];
            zachotka[0] = ocinka;
        } else {
            zachotka = Arrays.copyOf(zachotka, zachotka.length+1);
            zachotka[zachotka.length-1]= ocinka;
        }
    }

    public Student(String n, int kurs) {
        name = n;
        this.kurs=kurs;
        zachotka=null;
    }

    public Student(Student arg)
    {
        this.name = arg.name;
        this.kurs = arg.kurs;
        if(arg.zachotka!=null)
            this.zachotka=arg.zachotka.clone();

    }

    public Student()
    {
```

```java
        this("Ivanenko",1);
    }

    @Override
    public String toString() {
        return "Student{" +
                "name='" + name + '\'' +
                ", kurs=" + kurs +
                ", zachotka=" + Arrays.toString(zachotka) +
                '}';
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student student = (Student) o;
        return getKurs() == student.getKurs() &&
                getName().equals(student.getName()) &&
                Arrays.equals(zachotka, student.zachotka);
    }

    @Override
    public int hashCode() {
        int result = Objects.hash(getName(), getKurs());
        result = 31 * result + Arrays.hashCode(zachotka);
        return result;
    }

    public void Print() {
        System.out.println(toString());
    }

    @Override
    protected Object clone() throws CloneNotSupportedException
    {
        return super.clone();
    }

}

/*
Наступний проект демонструє ступінчастий масив,
передачу масива у функцію та повернення масиву з функції у якості
результата. Продемонстровано клонування масиву та перерахування
 */

package com.company;

import java.util.Arrays;
import java.util.Random;

enum Day {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY
}

public class Main {

    public static void main(String[] args) {
```

```
    // write your code here
        int [] arr= new int[]{4,2,7,6,2,8,9,10,23};

        int [][] arrrect= new int[5][];

        Random rnd = new Random();

        for( int i=0; i<arrrect.length; i++ ) {
            arrrect[i]= new int [3+rnd.nextInt(5)];
        }

//        for( int i=0; i<arrrect.length; i++ ) {
//
//            System.out.println(Arrays.toString(arrrect[i]));
//        }

        for( int i=0; i<arrrect.length; i++ ) {
            Print(arrrect[i]);
        }

        Print(arr);

        Print(func());


        int intArray[] = new int[3];
        byte byteArray[] = new byte[3];
        short shortsArray[] = new short[3];
        String[] strArray = new String[3];

        System.out.println(intArray.getClass());
        System.out.println(intArray.getClass().getSuperclass());
        System.out.println(byteArray.getClass());
        System.out.println(shortsArray.getClass());
        System.out.println(strArray.getClass());
        //-------------------------------
        int []intArray2 = {1,2,3};

        int cloneArray[] = intArray2.clone();

        cloneArray[0]=100;
        cloneArray[1]=200;
        cloneArray[2]=300;

        System.out.println( Arrays.toString(intArray2) );

        for( Day d : Day.values() )
            System.out.println(d);

    }

    public static void Print( int [] arg )
    {
        System.out.println("Print: "+Arrays.toString(arg));
    }

    public static int[] func()
    {
        // returning  array
        return new int[]{1,2,3};
```

```
    }
}


/*
Наступний проект Він демонструє
-робота з одновимірними масивами примітивних типів
-робота з одновимірними масивами посилальних типів
-сортування одновимірних масивів посилальних типів
 */
// Program.java
import java.util.*;
import java.util.Scanner;

public class Program {
    public static void main(String []args)throws CloneNotSupportedException{

//Одновимірний масив примітивного типу
        int [ ]arr;

        arr= new int[5];

//Створення масиву одночасно з ініціалізацією
        int [] ar2 = new int[]{3,2,1,2,3};

//Перший спосіб виведення одновимірного масиву
        for( int i=0; i<arr.length; ++i )
        {
            System.out.printf("%d ", arr[i]);
        }

        System.out.println();

//Другий спосіб виведення одновимірного масиву - foreach
        for( int e :ar2 )
            System.out.printf("%d ", e);

        System.out.println();

//Третій спосіб виведення одновимірного масиву - Arrays.toString
        String s="toString: "+java.util.Arrays.toString(ar2);
        System.out.println(s);

//Оригінальний масив посилального типу (два кроки створення)
        Student [] gruppa=new Student[6];

        gruppa[0]=new Student("BarakObama",2);
        gruppa[1]=new Student("Donaldtrump",5);
        gruppa[2]=new Student("GeorgeBush",1);
        gruppa[3]=new Student("Donaldtrump",4);
        gruppa[4]=new Student("Donaldtrump",4);
        gruppa[5]=new Student("BarakObama",3);

//Виведення масиву посилального типу через Arrays.toString
//можливе тому, що в класі Student заміщена функція
// @Override
// public String toString()
        s="Gruppa= "+Arrays.toString(gruppa);
        System.out.println(s);

        Student []copiedgruppa;
/*   //Стандартне - поверхневе - копіювання масиву
```

```
     //засобами Arrays.copyOf
     Student []copiedgruppa=
                java.util.Arrays.copyOf(
                        gruppa,gruppa.length);
*/

/*  //Інший спосіб копіювання - все одно поверхневий
        //copiedgruppa= gruppa.clone();
*/

//Глибинне копіювання масиву посилального типів
    copiedgruppa = new Student[gruppa.length];

        for(int i=0; i<gruppa.length; i++ )
        {
            copiedgruppa[i]= (Student)gruppa[i].clone();
        }

//Чи вплинуть оці команди на елементи масиву copiedgruppa?
        gruppa[0].kurs=10;
        gruppa[1].kurs=10;
        gruppa[2].kurs=10;

        s="Novostvorena Gruppa= "+java.util.Arrays.toString(copiedgruppa);
        System.out.println(s);

        //---------------------------------
        // Сортування масиву примітивних типів
        s="Original array of int: "+
                java.util.Arrays.toString(ar2);
        System.out.println(s);

        java.util.Arrays.sort(ar2);

        s="Sorted array of int: "+
                java.util.Arrays.toString(ar2);
        System.out.println(s);

        //------------------------------
        gruppa[0]=new Student("BarakObama",2);
        gruppa[1]=new Student("Donaldtrump",5);
        gruppa[2]=new Student("GeorgeBush",1);
        gruppa[3]=new Student("Donaldtrump",4);
        gruppa[4]=new Student("Donaldtrump",4);
        gruppa[5]=new Student("BarakObama",3);

        s="Original array of Students: "+
                java.util.Arrays.toString(gruppa);
        System.out.println(s);

        System.out.print("Enter sorting method (1- by course/another - by name):");
        Scanner sca = new Scanner(System.in);
        int sorting_method = sca.nextInt();

        if( sorting_method==1)Student.umova=true;
        else Student.umova=false;

// Сортування масиву посилальних типів можливе завдяки тому
// що клас Student реалізує інтерфейс Comparable<Student>
// об'єкти порівнюються функцією класу Student:
//public int compareTo(Student st2)
//метод сортування визначається статичною змінною Student.umova
```

```
        java.util.Arrays.sort(gruppa);

        s="Sorted array of Students: "+
                java.util.Arrays.toString(gruppa);
        System.out.println(s);

// Сортування масиву посилальних типів згідно курсу
//з використання компаратора Student.KursComparator
        java.util.Arrays.sort(gruppa, Student.KursComparator);

        s="Sorted by kurs of Students: "+
                java.util.Arrays.toString(gruppa);
        System.out.println(s);

// Сортування масиву посилальних типів згідно name
//з використання компаратора Student.NameComparator
        java.util.Arrays.sort(gruppa, Student.NameComparator);

        s="Sorted by name of Students: "+
                java.util.Arrays.toString(gruppa);
        System.out.println(s);
    }


}
```

// Student.java

```
import java.util.Arrays;
import java.util.Scanner;
import java.util.Comparator;


public class Student implements Cloneable, Comparable<Student> {

    String name;
    public int kurs;
    double SerednijBal;
    double [] zachotka;

    static String University="VNTU";

    public double SerednijBal()
    {
        this.SerednijBal=0.0;

        if( zachotka.length >0 ) {
            for (int i = 0; i < zachotka.length; i++)
                this.SerednijBal += zachotka[i];

            this.SerednijBal = this.SerednijBal / ((double) zachotka.length);
        }
        return this.SerednijBal;
    }

    public void SerednijBal(double ocinka )
    {
        if( zachotka.length >0 ) {
            for (int i = 0; i < zachotka.length;
                 i++)
```

```
                zachotka[i]=ocinka;
            }
        }

        //Порівняння двох об'єктів Student за змістом
        @Override
        public boolean equals(Object obj) {
            if (this == obj)
                return true;
            if (obj == null)
                return false;
            if (getClass() != obj.getClass())
                return false;
            Student other = (Student) obj;
            if (other.name != name)
                return false;
            if (other.kurs != kurs)
                return false;

            if( (other.zachotka==null)&&(zachotka==null))return true;
            if( (other.zachotka==null)||(zachotka==null))return false;
            if(other.zachotka.length!=zachotka.length)return false;

            for( int i=0; i<zachotka.length; i++)
            {
                if(other.zachotka[i]!=zachotka[i])
                    return false;
            }

            return true;
        }


        public Student( String n,
                        int kurs )
        {
            name = n;
            this.kurs = kurs;
            Student.University="Politeh";  //
            zachotka = null;

            System.out.printf("Конструктор створює об'єкт:%s %d\n", name, this.kurs);

            //for( int i=0; i<len; i++ )
            //    System.out.printf("%f ", zachotka[i]);
        }

        public void Exam(double ocinka) {
            if (zachotka == null) {
                zachotka = new double[1];
                zachotka[0] = ocinka;
            } else {
                zachotka = Arrays.copyOf(zachotka,
                        zachotka.length+1);
                zachotka[zachotka.length-1]= ocinka;
            }

        }
    // Конструктор по замовчуванню
        public Student()
        {
            //Делегування конструктора
```

```
        this("Ivanov",1);
    }


    //Поверхневе копіювання об'єкту Student
    @Override
    protected Object clone() throws CloneNotSupportedException
    {
        return super.clone();
    }

    /*
    //Глибинне копіювання об'єкту Student
    public Object clone()
    {
        Student tmp = new Student();
        tmp.name = name;
        tmp.kurs = kurs;
        tmp.zachotka= (zachotka==null)?null:
                Arrays.copyOf(zachotka, zachotka.length);

        return tmp;
    }
*/
    public void Print()
    {
        System.out.printf("Name:%s Kurs:%d SBal:%f\n",
                name, kurs, SerednijBal);
        if(zachotka!=null) {
            for (double a : zachotka)
                System.out.printf("%3.1f ", a);
        }
        else  System.out.println("Pusta zachotka!");

        System.out.println();
    }

    //Дозволяє перетворювати об'єкт класу Student
    //рядок
    @Override
    public String toString()
    {
        String rez= "["+name+"," +
                Integer.toString(kurs)+"]";
        return rez;
    }

    //Дозволяє змінювати критерій сортування
    public static boolean umova=false;

    //Реалізація методу порівняння об'єктів Student
    //з інтерфейсу Comparable<Student>
    public int compareTo(Student st2)
    {
        if( Student.umova )
        {
            if( this.kurs<st2.kurs )return -1;

            if( this.kurs>st2.kurs )return 1;

            return 0;
        }
```

```
        else return this.name.compareTo(st2.name);
    }

    /* // Приклад сортування, який використовує ДВА критерія
    public int compareTo(Student st2)
    {
        if( this.name.equals(st2.name) )
        {
            if( this.kurs<st2.kurs )return -1;

            if( this.kurs>st2.kurs )return 1;

            return 0;
        }
        else return this.name.compareTo(st2.name);
    }
*/


    //Компаратор для порівняння за курсом
    public static Comparator<Student> KursComparator
            = new Comparator<Student>() {

        @Override
        public int compare(Student e1, Student e2) {
            if (e1.kurs < e2.kurs) return -1;
            if (e1.kurs > e2.kurs) return 1;
            return 0;

        }
    };
    //Компаратор для порівняння за name
    public static Comparator<Student> NameComparator
                = new Comparator<Student>() {

            @Override
            public int compare(Student e1, Student e2) {
                return e1.name.compareTo(e2.name);

            }
        };

}



/*
Проект ілюструє відмінність вбудованих класів String та StringBuilder
Критерії порівняння -6 функцій, закоментованих у main.
 */
//Main.java
package com.company;

import java.util.Arrays;

public class Main
{
   private static final String STR = "A";

   public static String stringConcatenation()
   {
```

```
    String s = STR;
    for (int i = 0; i < Short.MAX_VALUE; i++) {
        s = s + STR;
    }
    return s;
}

public static String stringBufferConcatenation()
{
    StringBuilder sb = new StringBuilder(STR);
    for (int i = 0; i < Short.MAX_VALUE; i++) {
        sb.append(STR);
    }
    return sb.toString();
}

public static void performance()
{
    long start = System.nanoTime();
    stringConcatenation();
    long end = System.nanoTime();

    System.out.println("The time taken by string concatenation: "
            + (end - start) + "ns");

    start = System.nanoTime();
    stringBufferConcatenation();
    end = System.nanoTime();

    System.out.println("The time taken by StringBuilder concatenation: "
            + (end - start) + "ns");
}

public static void mutability() {
    String s = "AB";
    String scopy=s;
    s += "C";

    if( s==scopy ) System.out.println("s==scopy");
    else System.out.println("s!=scopy");


    StringBuilder sb=new StringBuilder("AB");
    StringBuilder sbcopy=sb;
    sb.append("C");

    if( sb==sbcopy ) System.out.println("sb==sbcopy");
    else System.out.println("sb!=sbcopy");

}

private static final String STR2 = "ABC";
public static void equality()
{
    String s1 = new String(STR);
    String s2 = new String(STR);
    System.out.println(s1.equals(s2));

    StringBuilder sb1 = new StringBuilder(STR);
    StringBuilder sb2 = new StringBuilder(STR);
    System.out.println(sb1.equals(sb2));
```

```
        //Since Java 11
        // System.out.println( sb1.compareTo(sb2) );
    }

    public static void constructor()
    {
        String str = "abc";

        StringBuilder sb= new StringBuilder("abc");
    }

    public static void comparable()
    {
        String[] colors = new String[]{"Red","Blue","green"};
        Arrays.sort(colors);
        System.out.println(Arrays.toString(colors));

        StringBuilder []codes= new StringBuilder[]{
                new StringBuilder("#008200"),
                new StringBuilder("#ff0000"),
                new StringBuilder("#006400")
        };

        try {
                Arrays.sort(codes);
        }
        catch (ClassCastException ex) {
            System.out.println(ex.getMessage());
        }
    }

    public static void length()
    {
        StringBuilder sb= new StringBuilder("abcdefgh");

        sb.setLength(5);
        System.out.println(sb);
        sb.setLength(10);
        System.out.println(sb);
        System.out.println((int)sb.charAt(9));

    }

    public static void main(String[] args)
    {

        mutability();
//        equality();
        //constructor();
//        comparable();
        //performance();
        //length();

    }

}
```

Контрольні питання

1. Як обробити елементи масиву з допомогою конструкції foreach?
2. Як порівнює об'єкти оператор ==?
3. Як здійснити порівняння об'єктів не за адресою а їх вмістом?
4. Для чого використовується функція public boolean equals(Object obj) ?
5. Чим відрізняється статичний елемент класу від нестатичного?
6. Чим відрізняється глибинне копіювання від поверхневого?
7. Як здійснити поверхневе копіювання?
8. Як здійснити глибинне копіювання?
9. Для яких класів достатньо поверхневого копіювання?
10. Чому поверхневе копіювання об'єктів класу String не призводить до проблем?
11. Для чого слід вказувати, що клас реалізує інтерфейс Cloneable, якщо він має функцію clone?
12. Для чого використовується функція Object.clone() і як вона викликається?
13. З якого класу викликається функція equals(), якщо вона не заміщена в класі користувача?
14. Як створити масив примітивних типів?
15. Як створити масив посилальних типів даних?
16. Як скопіювати одновимірний масив?
17. Для чого використовується функція Arrays.copyOf?
18. Для чого використовується функція Arrays.copyOfrange?
19. Для чого використовується функція System.arraycopy?
20. Для чого використовується функція Arrays.toString?
21. Для чого використовується виклик super.clone()?
22. Чому для копіювання масиву примітивних типів достатньо Object.clone() або Arrays.copyOf, а для масива посилальних типів недостатньо?
23. Як здійснити глибинне копіювання масива посилальних типів?
24. Як створити двовимірний масив примітивних типів?
25. Як створити двовимірний масив посилальних типів?

26. Як створити ступінчастий масив?

27. Як правильно склонувати двовимірний масив?

28. Як за значеннями порівняти двовимірний масив?

29. Яку функцію виконує Arrays.equals?

30. Яку функцію виконує Arrays.deepEquals?

31. В чому відмінність Arrays.equals від Arrays.deepEquals?

32. Яку функцію виконує Arrays.sort?

33. Як відсортувати масив примітивних типів?

34. Як відсортувати масив посилальних типів (reference type)?

35. Як реалізувати операцію порівняння двох об'єктів класу, створеного програмістом?

36. Для чого потрібен інтерфейс Comparable і як використовується?

37. Як інтерпретується ціле значення, яке повертає метод compareTo, який реалізується при використанні інтерфейсу Comparable?

38. Що означає термін “лексикографічне порівняння”?

39. До чого використовується компаратор Collections.reverseOrder() при сортуванні масива функцією Arrays.sort?

40. Як при використанні функції Arrays.sort вказати свою власну функцію для порівняння об'єктів?

41. Чим відрізняється алгоритм бінарного пошуку (binary search) від звичайного лінійного пошуку?

42. Яке максимальне число порівнянь можна зробити при бінарному пошуку числа серед масиву з N елементів? Алгоритм пошуку має або повернути індекс числа в масиві, або сказати, що цього числа в масиві нема.

43. Яка умова накладається на масив, в якому ведеться бінарний пошук?

44. Продемонструйте різницю між Arrays.toString и Arrays.deepToString

45. Що таке інтерфейс і для чого використовується?

46. Що означає термін "функціональний інтерфейс"? Наведіть приклад

47. Як клас реалізує інтерфейс?

48. Що таке компаратор і як він використовується?

49. Як побудувати і використати компаратор з використанням об'єкта статичного внутрішнього класу (nested class)?

50. Як побудувати і використати компаратор на основі об'єкта анонімного класу?

51. Як побудувати і використати компаратор на основі посилання (method reference) на статичний метод класу?

52. Що таке абстрактний клас? Які його властивості? Як позначити абстрактний клас?
53. Чим інтерфейс відрізняється від абстрактного класу?
54. Як створити інтерфейс і що це таке?
55. Які ключові слова використовуються при реалізації інтерфейсу?
56. Відмінність між String та StringBuilder: mutability.
57. Відмінність між String та StringBuilder: performance.
58. Як перевірити рівність двох String?
59. Як перевірити рівність двох StringBuilder?
60. Якщо в циклі робиться багато тисяч перетворень рядків, то який клас краще використовувати String чи StringBuilder?
61. Чому зберігання даних в непараметризованій колекції ArrayList не вигідно і краще використати параметризовану колекію ArrayList<E>?
62. Які переваги мають параметризовані колекції (generics) в порівнннні з непараметризованими?
63. Які переваги має масив в порівнянні зі списком?
64. Які переваги має список в порівнянні з масивом?
65. Як переглядати вміст колекції з допомогою ітератора?
66. Розкажіть про структуру даних стек. Який режим запису/зчитування даних?
67. Колекція Stack<> - будова, додання елементів, вилучення елементів, подивитись наступний елемент, пошук елементів, виведення вмісту на екран. Де може використовуватись колекція Stack<>?
68. Колекція Queue<> - будова, додання елементів, вилучення елементів, подивитись наступний елемент, перевірка наявності елементів, виведення вмісту на екран. Де може використовуватись колекція Queue<>?
69. Колекція PriorityQueue<> - будова, додання елементів, вилучення елементів, подивитись наступний елемент, перевірка наявності елементів, виведення вмісту на екран. Що треба зробити, щоб колекція PriorityQueue<> працювала з класом програміста? Де може використовуватись колекція PriorityQueue<>?
70. Колекція Deque<> - будова, додання елементів, вилучення елементів, подивитись наступний елемент, перевірка наявності елементів, виведення вмісту на екран з допомогою ітератора. Де може використовуватись колекція Deque<>?
71. Колекція Set<> - будова, додання елементів, вилучення елементів, виведення вмісту на екран, перевірка наявності елементів. Що треба зробити, щоб колекція Set<> працювала з класом програміста (і не дозволяла додання однакових за змістом об'єктів)?
72. Як зробити так, щоб колекція Set<> містила об'єкти не тільки унікальні, а ще й у відсортованому порядку?

73. Яка роль функцій hashCode() та equals(Object o) класу, який створив програміст у роботі колекції Set<>, яка зберігає об'єкти цього класу?
74. Колекція Set<>: продемонструвати об'єднання множин.
75. Колекція Set<>: продемонструвати перетинання множин.
76. Колекція Set<>: продемонструвати різницю (difference) множин.
77. Колекція Set<>: продемонструвати симеричну різницю (symmetric difference) множин.
78. Колекція Map<> - будова, додання елементів, вилучення елементів, доступ до елемента по ключу (Key), зміна значення (Value) в парі, перегляд множини пар entrySet(), доступ до множини ключів keySet() та колекції значень values(), виведення вмісту колекції на екран, перевірка наявності ключа containsKey(Key) та значення containsValue(Value).
79. Використання методу forEach для обробки елементів Set<>, List <> та Map<>.
80. Використання методу removeIf для обробки елементів Collection.
81. Використання методу compute для обробки елементів Map.
82. Використання методу computeIfAbsent для обробки елементів Map.
83. Використання методу computeIfPresent для обробки елементів Map.
84. Використання методу getOrDefault для обробки елементів Map.
85. Використання методу merge для обробки елементів Map.
86. Використання методу putIfAbsent для обробки елементів Map.
87. Використання методу replaceAll для обробки елементів Map.
88. Колекція LinkedList<>: - будова, додання елементів, вилучення елементів, зміна значення елементів.
89. Продемонструвати ситуацію, в якій LinkedList<> працює ефективніше за ArrayList<>.
90. Продемонструвати ситуацію, в якій ArrayList<> працює ефективніше за LinkedList<>.